RAPPORT DE GESTION DE PROJET

Du POC au MVP : étude de conception de l'assistant RAG Puls-Events

Recommandation d'événements culturels à Brest : mémoire conversationnelle, contexte géographique, recherche web temps réel et monitoring

Élément	Détail
Projet	Assistant de recommandation d'événements culturels, Puls-Events (passage du POC au MVP)
Auteur	Daniel, Data Engineer en alternance
Destinataire	Jérémy, responsable technique Puls-Events
Date et version	23 octobre 2026, version 1.0 (étude de conception à valider au jalon J0)
Documents liés	Classeur Excel « MVP_Puls-Events_backlog_couts_planning.xlsx » (backlog, risques, jalons, planning, build, OPEX, optimisations, hypothèses), support de présentation

1. Introduction

Contexte du projet

Puls-Events est une plateforme qui veut simplifier la recherche de sorties culturelles pour les habitants de Brest. Dans un agenda en ligne classique, trouver un événement qui plaît demande de parcourir de nombreuses pages. Un premier projet, le **POC**, a démontré qu'un assistant conversationnel pouvait comprendre une question en langage naturel et recommander directement les bons événements.

Le POC repose sur une architecture **RAG** (Retrieval-Augmented Generation) pour éviter que le modèle n'invente des événements : les données réelles d'OpenAgenda sont récupérées, nettoyées, découpées en extraits (chunks de 500 caractères avec un recouvrement de 50), transformées en vecteurs avec **mistral-embed**, indexées dans **FAISS**, puis interrogées par une chaîne **LangChain** qui force **Mistral Small** à répondre uniquement à partir des extraits retrouvés. Le POC a été validé : 697 événements de Brest sur la dernière année, environ 1 245 extraits vectorisés, une recherche dédupliquée par identifiant d'événement et quatre tests automatisés réussis.

Objectif du projet

Jérémy demande maintenant une **étude de conception du MVP** (Minimum Viable Product) : un assistant utilisable par de vrais utilisateurs, déployé dans le cloud et capable de monter en charge. Quatre axes techniques doivent être couverts :

1. **Mémoire conversationnelle** : l'assistant se souvient des échanges précédents pour affiner ses réponses.
2. **Contexte géographique** : les recommandations tiennent compte de la position ou du quartier de l'utilisateur.
3. **Recherche web en temps réel** avec la bibliothèque Hugging Face **smolagents**, pour les informations absentes de la base.
4. **Suivi des performances** (monitoring) : qualité des réponses, disponibilité, fraîcheur des données et coûts.

Ce document suit le plan du template de rapport de gestion de projet : analyse des besoins, plan de projet, macro backlog, architecture technique détaillée (avec le choix du cloud justifié par une veille), estimation des coûts de construction et d'exploitation, bilan, conclusion et annexes. Les tableaux chiffrés (backlog, coûts)

existent aussi au format Excel, recalculables à partir d'hypothèses modifiables, pour que Jérémy puisse tester d'autres scénarios.

> **Ce que ce document ne fait pas** : il ne construit pas le MVP. Il décrit quoi faire, avec quoi, en combien de temps et pour quel budget. Tous les chiffres sont des estimations de conception, avec leurs hypothèses visibles dans le classeur Excel.

2. Analyse et synthèse des besoins formulés par l'équipe

2.1 Synthèse du contexte, objectifs métiers et contraintes techniques

Besoins formulés et ce qu'ils impliquent

Axe demandé	Besoin métier	Implication technique pour le MVP
Mémoire conversationnelle	Pouvoir enchaîner : « et c'est gratuit ? », « plutôt pour des enfants »	Identifiant de session, fenêtre glissante sur les 5 derniers échanges, reformulation de la question en question autonome avant la recherche, stockage des sessions et politique de rétention RGPD
Contexte géographique	Proposer des événements proches du lieu de l'utilisateur	Coordonnées des événements dans la base, calcul de distance (PostGIS), saisie du quartier ou géolocalisation avec consentement
Recherche web en temps réel	Répondre quand la base ne suffit pas (information récente, précision manquante)	Agent smolagents isolé, déclenché par un routeur seulement en repli, avec garde-fous (étapes, durée, outils, coût) et mention de l'origine web
Suivi des performances	Savoir si l'assistant est fiable, disponible, à jour et abordable	Métriques techniques, évaluation Ragas régulière sur un jeu de test annoté par un humain, fraîcheur de l'index, suivi des coûts

Objectifs métiers et indicateurs de réussite

Objectif métier	Indicateur (cible proposée, à valider)	Mode de mesure
Réponses fiables, sans invention	Fidélité (faithfulness Ragas) cible à fixer après la première mesure ; 0 événement inventé sur le jeu de test	Évaluation Ragas hebdomadaire
Réponses pertinentes	Pertinence de la réponse et précision du contexte suivies dans le temps (tendance stable ou en hausse)	Ragas : answer_relevancy, context_precision, context_recall
Expérience fluide	Latence p95 inférieure à 6 secondes ; disponibilité mensuelle d'au moins 99 %	CloudWatch (latence, erreurs 5xx)
Données à jour	Index mis à jour chaque jour, fraîcheur inférieure à 24 h	Journal d'ingestion, alerte en cas d'échec
Coûts maîtrisés	Coût par requête inférieur à 0,010 € dès 10 000 utilisateurs actifs (estimation : 0,005 €)	Suivi des tokens et AWS Budgets

Contraintes techniques et réglementaires

- **Continuité avec le POC** : LangChain et Mistral sont conservés pour ne pas réécrire ce qui fonctionne.
- **Qualité des données OpenAgenda** : dans les 697 événements du POC, 508 (73 %) sont des offres d'emploi France Travail et environ 189 seulement relèvent de la culture. Un filtrage par catégorie est indispensable avant toute montée en charge.

- **RGPD** : les conversations peuvent contenir des données personnelles et la géolocalisation est une donnée sensible. Pas de compte utilisateur au MVP, sessions anonymes, purge à 30 jours, consentement explicite avant toute géolocalisation.
- **Sécurité** : clés API hors du code (leçon du POC), agent web exécuté dans un conteneur isolé, limitation de débit pour éviter les abus et les coûts imprévus.
- **Latence et coût** : chaque étape supplémentaire (mémoire, recherche web) ajoute du temps et des tokens ; elles sont donc bornées par des plafonds.
- **Évaluation humaine** : conformément à la consigne du mentor, les réponses attendues du jeu de test sont rédigées par une personne, pas par une IA.
- **Équipe réduite** : une architecture simple, avec peu de services à exploiter, est préférée à une architecture sophistiquée.

2.2 Analyse des utilisateurs cibles et des cas d'usage

Utilisateur cible	Besoin principal	Ce que le MVP lui apporte
Habitant de Brest (sortie spontanée)	Trouver vite quoi faire ce soir ou ce week-end, près de chez lui, souvent gratuit	Recherche par proximité, réponses courtes avec lieu, date et conditions, conversation qui s'affine
Parent ou famille	Événements adaptés aux enfants, créneaux et tarifs	Mémoire (« et pour un enfant de 6 ans ? »), prise en compte de l'âge et du prix
Visiteur ou nouvel arrivant	Ne connaît pas la ville ni les quartiers	Questions par quartier ou adresse, informations récentes via la recherche web (changement d'horaire, annulation)
Équipe Puls-Events (Jérémy)	Savoir si l'assistant est fiable, à jour et abordable	Tableaux de bord techniques, scores Ragas, alertes, suivi du coût par requête

ID	Cas d'usage	Exemple concret	Axe	Priorité
UC1	Recommandation simple	« Quels concerts ce week-end à Brest ? »	Socle (POC)	Must-Have
UC2	Affiner la conversation	« Et c'est gratuit ? » juste après une réponse	Mémoire	Must-Have
UC3	Chercher près de soi	« Une exposition près de Recouvrance, à moins de 2 km »	Géographie	Must-Have
UC4	Information récente hors base	« Le concert de ce soir est-il maintenu ? »	Recherche web	Must-Have
UC5	Refus propre hors périmètre	« Quel temps fait-il ? », « Événements à Paris »	Socle (POC)	Must-Have
UC6	Suivre la qualité	Jérémy consulte les scores Ragas et les alertes	Monitoring	Must-Have
UC7	Suivre coûts et fraîcheur	Dernière ingestion, tokens consommés, coût par requête	Monitoring	Must-Have
UC8	Donner son avis	Pouce haut ou bas sur une réponse	Monitoring	Nice-to-Have

3. Plan de projet

Le projet est organisé en **12 semaines de travail**, démarrant le lundi 2 novembre 2026 après validation de l'étude (jalon J0), avec une pause de deux semaines (21 et 28 décembre) qui décale la fin au vendredi 5 février 2027. La méthode est incrémentale : cycles de deux semaines, une démonstration à Jérémy à chaque jalon, et un registre de risques revu chaque semaine.

3.1 Jalons

Jalon	Intitulé	Date cible	Livrables	Critère d'acceptation
J0	Validation de l'étude de conception du MVP	30/10/2026	Rapport de gestion de projet, classeur Excel (backlog, coûts, planning), support de présentation	Validation par Jérémy (responsable technique) et accord sur le périmètre Must-Have
J1	Socle cloud et CI/CD opérationnels	13/11/2026	Code Terraform, pipeline GitHub Actions, environnements dev et staging	Déploiement d'une API de test en staging depuis un commit, sans action manuelle
J2	Données industrialisées et base vectorielle pgvector	27/11/2026	Ingestion quotidienne automatisée, filtrage culturel, base PostgreSQL (pgvector + PostGIS)	Ingestion réussie 5 jours de suite, pertinence au moins égale à celle du POC sur le jeu de test
J3	API RAG avec mémoire et contexte géographique	18/12/2026	API FastAPI, mémoire sur 5 échanges, recherche par proximité, politique RGPD, tests automatisés	Scénarios de conversation de la recette passés, latence p95 inférieure à 6 s en staging
J4	Agent web smolagents et interface utilisateur	15/01/2027	Agent de repli (sandbox, plafonds), routeur RAG / web, interface Streamlit	Les questions hors base reçoivent une réponse étiquetée « web » ou un refus clair, sans dépasser le plafond d'appels
J5	Monitoring et évaluation continue en place	22/01/2027	Tableaux de bord CloudWatch, alarmes, évaluation Ragas hebdomadaire, historique des scores	Première évaluation Ragas exécutée automatiquement, alarmes testées
J6	Recette et démonstration du MVP	29/01/2027	Rapport de recette, résultats des tests de charge, sécurité validée	Aucune anomalie bloquante, test de charge réussi sur le scénario Croissance
J7	Mise en production du MVP et bilan	05/02/2027	MVP en production, documentation et runbook, bilan du projet	MVP accessible aux utilisateurs pilotes, équipe Puls-Events formée

3.2 Échéanciers

Le diagramme ci-dessous place chaque lot de travail sur le calendrier. Les losanges rouges sont les jalons ; la zone grise est la pause de fin d'année.

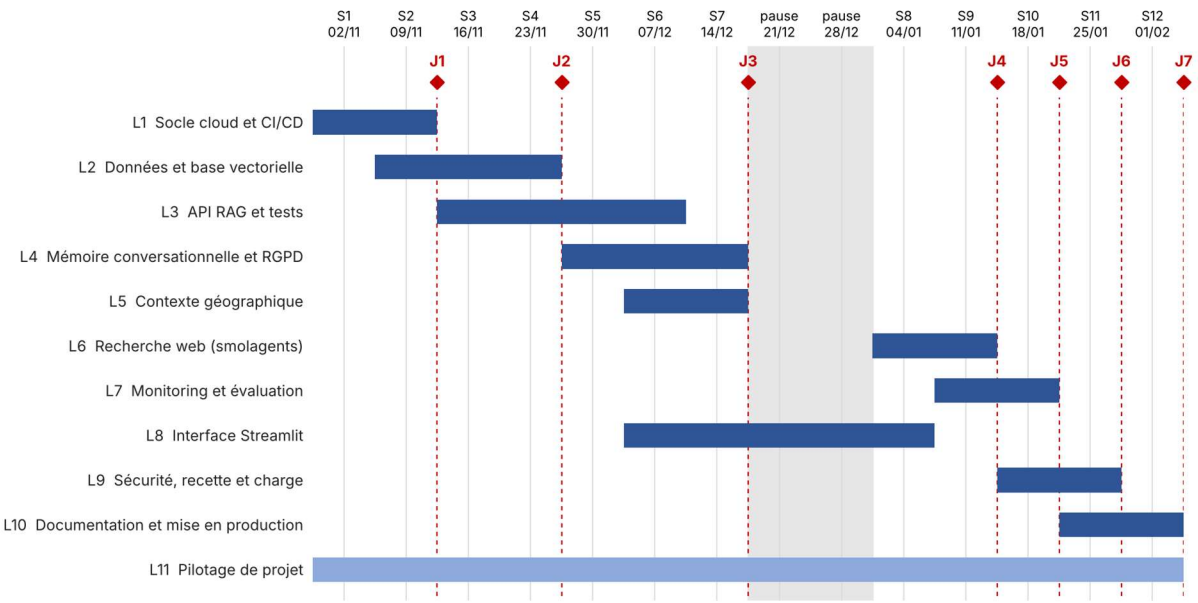


Figure 1 : planning macro du MVP (S1 = semaine du 2 novembre 2026)

Lot	Intitulé	Semaines	Charge (jours)	Coût estimé
L1	Socle cloud et CI/CD	S1 à S2	10	5 180 €
L2	Données et base vectorielle	S2 à S4	16	6 910 €
L3	API RAG et tests	S3 à S6	13	6 040 €
L4	Mémoire conversationnelle et RGPD	S5 à S7	8	3 720 €
L5	Contexte géographique	S6 à S7	8	3 640 €
L6	Recherche web (smolagents)	S8 à S9	10	4 760 €
L7	Monitoring et évaluation	S9 à S10	8	3 700 €
L8	Interface Streamlit	S6 à S8	6	3 000 €
L9	Sécurité, recette et charge	S10 à S11	10	5 020 €
L10	Documentation et mise en production	S11 à S12	5	2 400 €
L11	Pilotage de projet	S1 à S12	8	4 800 €
	Total		**102**	**49 170 €**

Hypothèses de calendrier : le projet suppose un développeur back-end à temps partiel en plus du Data Engineer (34 jours sur 12 semaines) et un appui DevOps ponctuel (11 jours). Sans ce renfort, Daniel absorbe la charge et la durée passe de 12 à environ 16 semaines (risque R08).

3.3 Livrables

Livrable	Jalon	Format	Responsable
Rapport de gestion de projet, classeur Excel, support de présentation	J0	Word, Excel, PowerPoint	Daniel
Code Terraform et pipeline CI/CD	J1	Dépôt GitHub	DevOps, Daniel
Ingestion automatisée et base PostgreSQL (pgvector, PostGIS)	J2	Dépôt, documentation des tables	Data Engineer
API RAG avec mémoire et recherche par proximité, tests automatisés	J3	Dépôt, rapport de tests	Développeur, Data Engineer
Agent web smolagents, routeur et interface Streamlit	J4	Services conteneurisés	Développeur
Tableaux de bord, alarmes, évaluation Ragas hebdomadaire	J5	CloudWatch, rapports Ragas (CSV)	Data Engineer, DevOps
Rapport de recette et résultats des tests de charge	J6	Document PDF ou Word	Data Engineer, Jérémy
MVP en production, runbook, bilan du projet	J7	Service en ligne, documentation	Toute l'équipe

4. Macro backlog des fonctionnalités

Le backlog liste les fonctionnalités par axe, avec une priorité **Must-Have** (indispensable au MVP, chiffrée dans le budget) ou **Nice-to-Have** (utile mais reportable, chiffrée en option). La complexité reflète le degré d'incertitude technique : Faible (technique maîtrisée), Moyenne (technique connue, intégration à faire) ou Élevée (technologie nouvelle pour l'équipe ou risque technique important). Le détail complet (jours par profil, dépendances, risque et mitigation de chaque ligne) est dans l'onglet « Macro backlog » du classeur Excel.

4.1 Fonctionnalités essentielles pour le MVP (Must-Have)

ID	Fonctionnalité	Axe	Complexité	Jours	Jalon
F01	Infrastructure as Code (Terraform) : VPC, ECS Fargate, ECR, RDS, S3, IAM	Socle	Moyenne	5	J1
F02	Pipeline CI/CD (GitHub Actions) : build, tests, déploiement ECS	Socle	Moyenne	4	J1
F03	Migration FAISS vers PostgreSQL (pgvector + PostGIS) : schéma événements, chunks, vecteurs	Données	Élevée	6	J2
F04	Filtrage des événements culturels (exclure les offres d'emploi France Travail)	Données	Faible	2	J2
F05	Ingestion quotidienne automatisée depuis OpenAgenda (incrémentale, Kestra)	Données	Élevée	7	J2
F06	Refonte du chatbot en API FastAPI (/chat, /health, schémas de données)	Socle	Moyenne	8	J3
F07	Tests automatisés (unitaires, intégration) et jeu de test de référence	Monitoring	Moyenne	4	J3
F08	Mémoire conversationnelle : identifiant de session, fenêtre des 5 derniers échanges, reformulation de la question	Mémoire	Moyenne	5	J3
F09	Politique de rétention RGPD des conversations (anonymisation, purge à 30 jours)	Mémoire	Faible	2	J3
F10	Coordonnées géographiques des événements (latitude, longitude) et géocodage de secours	Géographie	Faible	2	J3
F11	Recherche par proximité : « près de moi » ou par quartier (filtre de distance et tri)	Géographie	Moyenne	5	J3
F12	Agent de recherche web temps réel (Hugging Face smolagents) en solution de repli	Recherche web	Élevée	6	J4
F13	Routeur RAG / web et garde-fous (seuil de pertinence, plafond d'appels, cache)	Recherche web	Élevée	3	J4
F14	Monitoring technique : CloudWatch, alarmes, latence p95, erreurs, coût des tokens	Monitoring	Moyenne	4	J5
F15	Évaluation continue de la qualité (Ragas hebdomadaire, historique des scores)	Monitoring	Moyenne	3	J5
F16	Interface Streamlit : chat, historique de la session, saisie de la localisation	Socle	Moyenne	5	J4
F17	Sécurité : Secrets Manager, WAF, limitation de débit, HTTPS	Socle	Moyenne	4	J6
F18	Recette utilisateur et tests de charge	Socle	Moyenne	4	J6
F19	Documentation (runbook, schéma, README) et mise en production	Socle	Faible	3	J7

Total Must-Have : **19 fonctionnalités, 82 jours** (Data Engineer 37 j, développeur 34 j, DevOps 11 j), hors pilotage de projet.

4.2 Fonctionnalités reportables (Nice-to-Have)

ID	Fonctionnalité	Axe	Complexité	Jours	Jalon
N01	Retour utilisateur (pouces haut / bas) exploité dans le suivi qualité	Monitoring	Faible	3	Post-MVP
N02	Traçage des appels LLM (Langfuse) et tableaux de bord	Monitoring	Moyenne	4	Post-MVP
N03	Cache sémantique des réponses	Socle	Moyenne	4	Post-MVP
N04	Recherche hybride (mots-clés + vecteurs) et re-classement des résultats	Données	Élevée	6	Post-MVP
N05	Compte utilisateur et préférences (goûts, quartier favori)	Socle	Élevée	7	Post-MVP

ID	Fonctionnalité	Axe	Complexité	Jours	Jalon
N06	Extension à d'autres villes	Données	Moyenne	5	Post-MVP
N07	Agent web multi-étapes avec citation des sources	Recherche web	Élevée	4	Post-MVP
N08	Montée en charge avancée : auto-scaling fin et Multi-AZ complet	Socle	Moyenne	3	Post-MVP
N09	Alertes ou newsletter d'événements personnalisée	Socle	Moyenne	4	Post-MVP

Total Nice-to-Have : **9 fonctionnalités, 40 jours**, soit environ 18 660 € en option, à décider après les premiers retours des utilisateurs pilotes.

4.3 Complexité, délais, risques et solutions de mitigation

Les estimations de délai incluent le temps de test et de correction, plus une marge de 15 % dans le budget. Les risques ci-dessous sont classés par criticité (probabilité de 1 à 3 multipliée par impact de 1 à 3). Les risques propres à une fonctionnalité sont aussi détaillés dans l'onglet « Macro backlog ».

ID	Risque	Criticité	Mitigation
R01	L'agent smolagents exécute du code non maîtrisé ou renvoie une information fausse	6 (Élevée)	Exécution isolée (sandbox), outils limités à la recherche web, étapes et durée plafonnées, mention de la source web dans la réponse
R05	Données personnelles dans les conversations (RGPD)	6 (Élevée)	Pas de compte, sessions anonymes, purge à 30 jours, information dans l'interface, avis du référent RGPD
R06	Qualité des données : événements non culturels, doublons, lieux hors périmètre	6 (Élevée)	Filtrage culturel, déduplication par uid, test métier automatisé, suivi du % d'événements retenus
R08	Ressources de développement non disponibles (profil développeur à temps partiel)	6 (Élevée)	Hypothèse à valider dès le cadrage ; à défaut, planning étendu de 12 à environ 16 semaines
R02	Le coût de l'API de recherche web dépasse le budget	4 (Moyenne)	Routeur (web seulement si nécessaire), plafond quotidien, cache des recherches, alerte AWS Budgets / fournisseur
R03	Latence trop élevée (embedding + LLM + éventuelle recherche web)	4 (Moyenne)	Objectif p95 inférieur à 6 s, réponse en streaming, délais maximum par étape, mesure dès la v1
R09	Dérive de la qualité des réponses après une mise à jour (données, prompt, modèle)	4 (Moyenne)	Évaluation Ragas hebdomadaire, jeu de test versionné, suivi des tendances
R10	Dépassement des coûts cloud (ressources fixes oubliées, NAT, trafic)	4 (Moyenne)	AWS Budgets avec alerte, revue mensuelle, optimisations de l'onglet Optimisations
R11	Calendrier : 4 axes en 12 semaines avec une pause de fin d'année	4 (Moyenne)	Priorisation Must-Have / Nice-to-Have, livraison par jalons, marge de 15 % dans le budget
R04	Quotas ou indisponibilité de l'API OpenAgenda	2 (Faible)	Ingestion incrémentale, reprise sur erreur, conservation du dernier index valide, alerte
R07	Dépendance à un fournisseur de LLM (Mistral)	2 (Faible)	Abstraction LangChain : changement de modèle par configuration, modèles Mistral aussi accessibles via les plateformes cloud

5. Architecture technique détaillée

5.1 Schéma d'architecture cloud scalable

L'architecture reprend les briques du POC (LangChain, Mistral) et remplace ce qui ne passe pas à l'échelle : l'index FAISS local devient une base PostgreSQL managée, le script en ligne de commande devient une API, et l'ingestion manuelle devient un traitement planifié. Tout tourne dans la région AWS de Paris (eu-west-3).

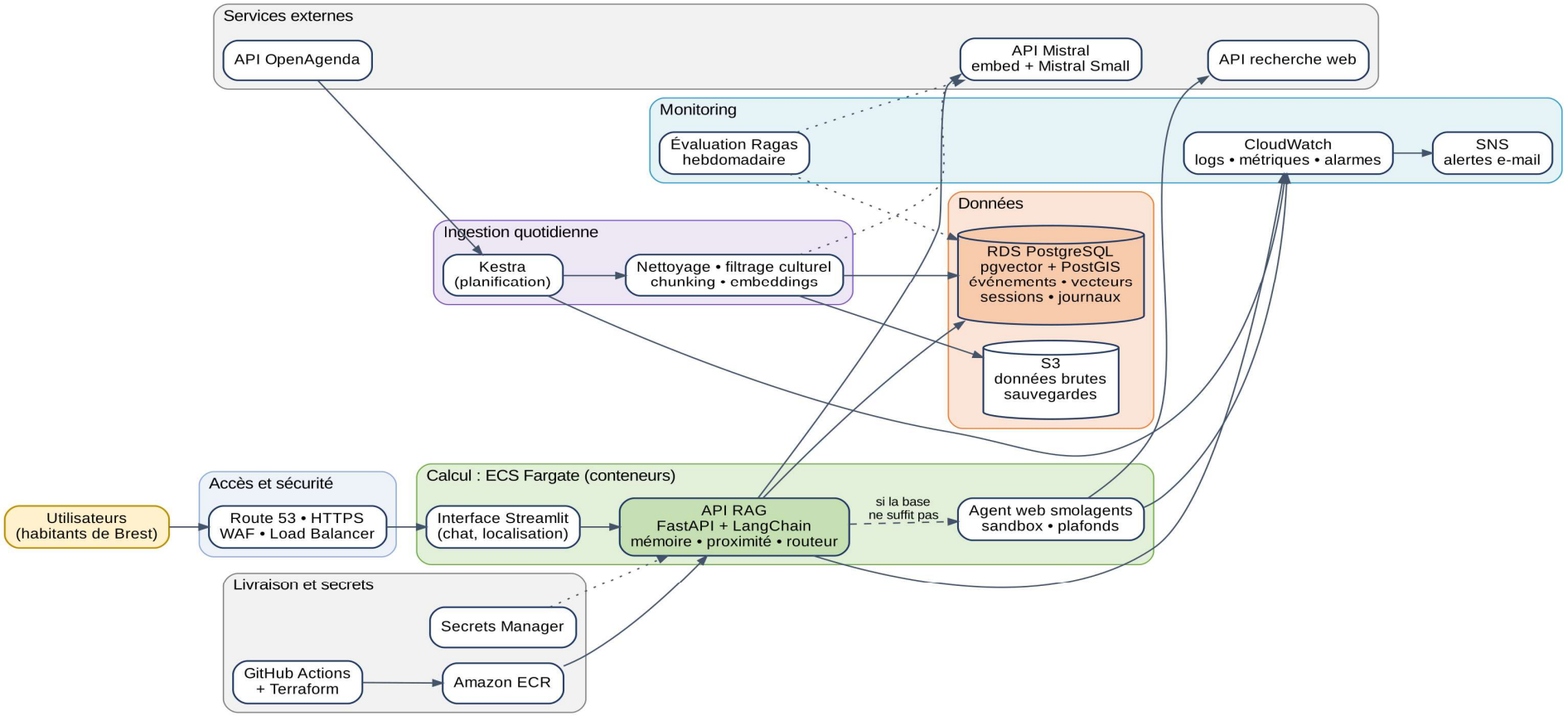


Figure 2 : architecture cible du MVP sur AWS (flux pointillés = appels conditionnels ou optionnels)

Comment une question traverse le système

1. L'utilisateur écrit dans l'interface **Streamlit**, derrière Route 53, le pare-feu **WAF** (limitation de débit) et un **Application Load Balancer**.
2. L'**API RAG** (FastAPI + LangChain) récupère l'historique des 5 derniers échanges de la session et reformule la question de façon autonome.
3. La question est vectorisée par **mistral-embed**, puis **PostgreSQL** (pgvector pour la similarité, PostGIS pour la distance) retourne les extraits les plus proches, dédupliqués par événement.
4. Si les extraits suffisent, **Mistral Small** rédige la réponse. Sinon, le routeur appelle l'**agent web smolagents**, isolé dans son propre conteneur, qui interroge une API de recherche et renvoie un résultat étiqueté comme issu du web.
5. La réponse, la latence, les tokens et la session sont journalisés (CloudWatch et table d'interactions), ce qui alimente le monitoring.

Composants, bases de données, services et API

Couche	Service	Rôle	Scalabilité et choix
Calcul	Amazon ECS Fargate (API, interface, agent web, Kestra)	Exécute les conteneurs sans serveur à gérer	Nombre de tâches ajusté par charge (de 2 à 6 pour l'API selon le scénario)
Base de données	Amazon RDS PostgreSQL avec pgvector et PostGIS	Événements, chunks et vecteurs, coordonnées, sessions de conversation, journal d'interactions, scores Ragas	Instance simple au pilote, Multi-AZ en croissance, taille supérieure à l'échelle ; sortie possible vers S3 Vectors au-delà de plusieurs millions de vecteurs
Stockage objet	Amazon S3	Données brutes OpenAgenda, sauvegardes, exports	Capacité illimitée, coût proportionnel au volume
Ingestion	Kestra + traitement Python	Collecte quotidienne, nettoyage, filtrage culturel, chunking, embeddings, mise à jour de l'index	Planifié ; remplaçable par EventBridge Scheduler (optimisation O1)
Modèles	API Mistral (mistral-embed, Mistral Small)	Vectorisation et génération des réponses	Facturation à l'usage ; modèle interchangeable via LangChain
Accès et sécurité	Route 53, WAF, ALB, Secrets Manager	HTTPS, limitation de débit, répartition de charge, gestion des clés	Géré par AWS, coûts faibles et stables
Observabilité	CloudWatch, SNS, tâche Ragas planifiée	Logs, métriques, alarmes, évaluation hebdomadaire	Seuils d'alerte limités à l'essentiel
Livraison	GitHub Actions, Amazon ECR, Terraform	Tests, build des images, déploiement reproductible	Environnements dev, staging, prod identiques

Contrat d'API exposé par l'API RAG (exemple simplifié) :

```
POST /chat
{ "session_id": "a1b2...", "message": "Et c'est gratuit ?", "location": {"lat": 48.39, "lon": -
4.49} }

200 OK
{ "answer": "...", "events": [{"title": "...", "date": "...", "place": "...", "distance_km":
1.2}],
  "source": "base" | "web", "latency_ms": 2840 }

GET /health  ->  200 si la base, le modèle et l'index répondent
```

5.2 Choix du cloud provider et justification par la veille technique

La veille a été menée le 15 octobre 2026 sur trois sujets : le marché des clouds, les options de stockage vectoriel, et les outils d'agent web. Les sources sont listées en fin de section et dans les annexes.

Comparaison des trois fournisseurs

Critère	AWS	Microsoft Azure	Google Cloud
Part du marché mondial (T2 2026, Synergy Research)	28 %	20 %	15 %
Région en France	Paris (eu-west-3)	France Central (Paris)	Paris (europe-west9)
Conteneurs sans serveur	ECS Fargate	Container Apps	Cloud Run
PostgreSQL managé avec pgvector	RDS et Aurora PostgreSQL	Azure Database for PostgreSQL	Cloud SQL et AlloyDB
Stockage vectoriel natif	S3 Vectors (disponible depuis déc. 2025), OpenSearch Serverless	Azure AI Search	Vertex AI Vector Search
Modèles Mistral	API directe (utilisée) ; aussi via Bedrock	API directe ; catalogue de modèles Azure	API directe ; catalogue Vertex AI
Expérience de l'équipe	Oui : Forecast 2.0 GreenAndCoop (RDS, ECS Fargate, ECR, EventBridge, CloudWatch)	Non	Non
Coût pour ce projet	Dominé par des ressources fixes (base, calcul, réseau) dont les prix sont comparables ; aucun écart décisif démontré entre les trois	Idem	Idem

Les équivalents Azure et Google Cloud existent pour chaque brique : la décision ne se joue pas sur une fonctionnalité manquante. Elle se joue sur trois critères : **l'expérience déjà acquise par l'équipe sur AWS** (moins de risque et de temps de montée en compétence sur 12 semaines), **le leadership du marché**, qui garantit un large écosystème et de nombreuses ressources, et **la possibilité de rester portable** : conteneurs, PostgreSQL et Terraform se transposent chez un autre fournisseur si besoin.

Veille sur le stockage vectoriel

Option	Modèle de coût	Forces	Limites pour ce projet
FAISS local (POC)	Gratuit	Rapide, simple, aucune infrastructure	Index sur disque d'une machine, pas de mise à jour continue, pas de requête géographique, pas d'accès partagé
PostgreSQL pgvector + PostGIS sur RDS (retenu)	Instance RDS d'environ 53 $/mois à Paris (db.t4g.medium)	Un seul endroit pour vecteurs, coordonnées, sessions et journaux ; filtres combinés en SQL	Coût fixe même sans trafic ; à surveiller au-delà de plusieurs millions de vecteurs
Amazon S3 Vectors	Stockage, écritures, requêtes et données traitées facturés à l'usage ; AWS annonce jusqu'à 90 % d'économie face à une base vectorielle spécialisée	Jusqu'à 2 milliards de vecteurs par index, environ 100 ms en usage fréquent, aucun serveur	Filtrage sur métadonnées mais pas de calcul de distance géographique ; surdimensionné pour environ 1 000 à 10 000 vecteurs
OpenSearch Serverless	Facturation par unités de calcul (OCU) et stockage	Recherche vectorielle, textuelle et géographique	Coût d'entrée plus élevé et mise en route plus lourde pour un volume aussi faible
Pinecone Serverless	Stockage et lectures/écritures, minimum de 50 $/mois en offre payante	Service dédié, très simple à utiliser	Brique supplémentaire à exploiter, hors AWS, sans gestion géographique native

Conclusion de la veille : au volume prévu (de l'ordre du millier à la dizaine de milliers de vecteurs), pgvector largement suffisant, et surtout il permet de combiner similarité, distance géographique et mémoire de session dans la même requête. S3 Vectors reste la voie de sortie si le volume explose.

Autres choix issus de la veille

- **LLM** : API Mistral conservée. Tarif relevé en septembre 2026 : Mistral Small 0,15 $ par million de tokens en entrée et 0,60 $ en sortie ; Mistral Large 0,50 $ et 1,50 $ (utilisé uniquement comme juge Ragas). Hébergement européen, cohérent avec le RGPD.

- **Agent web** : la bibliothèque smolagents (Hugging Face) est légère, compatible avec de nombreux modèles (dont Mistral via LiteLLM) et propose des agents qui écrivent leurs actions en code, avec des options d'exécution isolée. Le moteur de recherche gratuit livré par défaut convient au test mais pas à la production : une API de recherche payante est prévue (estimée à 0,004 $ par recherche, à confirmer par devis).
- **Orchestration** : Kestra, déjà maîtrisé sur un projet précédent (BottleNeck) et réutilisé depuis. Une alternative plus légère (EventBridge Scheduler) est chiffrée dans les optimisations.

> **Décision : AWS, région de Paris.** Expérience de l'équipe, écosystème le plus large, services managés couvrant tous les besoins, données hébergées en France.

Sources de la veille :

- Parts de marché T2 2026 (Synergy Research, relayé par) : Artic Sledge, Cloud Market Share 2026
- Amazon S3 Vectors disponible en général : AWS News Blog
- Comparatif des options vectorielles sur AWS : RDarrylR/aws-vector-hosting-comparison (GitHub)
- Tarifs Mistral API : CloudZero, Mistral API pricing
- Prix RDS db.t4g.medium : Bytebase, db.t4g.medium pricing
- Agents smolagents : Hugging Face, smolagents guided tour
- Dépôt smolagents : github.com/huggingface/smolagents

5.3 Stratégie de déploiement, de modularité et de monitoring

Déploiement

Principe	Mise en œuvre
Infrastructure as Code	Tout le socle (réseau, ECS, RDS, S3, droits) décrit en Terraform et versionné sur GitHub : un environnement se recrée en une commande
Trois environnements	dev (local et AWS réduit), staging (identique à la prod, taille réduite), prod ; mêmes images de conteneurs, seules les variables changent
Pipeline CI/CD	GitHub Actions : vérification du code, tests automatisés, construction des images, publication dans ECR, déploiement en staging, test de fumée avec un mini-jeu de questions, validation manuelle, déploiement en prod
Déploiement sans coupure	Mise à jour progressive des tâches ECS avec retour arrière automatique si le contrôle de santé échoue
Données	Migrations de base versionnées ; index reconstruit en parallèle puis basculé ; dernier index valide conservé si l'ingestion échoue
Secrets	Clés Mistral et recherche web dans Secrets Manager, jamais dans le code ni dans les images

Modularité

- **Quatre services indépendants** : ingestion, API RAG, agent web et interface. Chacun a son conteneur, ses tests et son cycle de déploiement.
- **Interfaces stables** : le retriever (base vectorielle), le modèle de langage et la mémoire sont derrière des abstractions LangChain. Changer de modèle, passer de pgvector à S3 Vectors ou remplacer Streamlit par une application web se fait sans toucher au reste.
- **Configuration par variables d'environnement** : modèle, nombre d'extraits, taille de la fenêtre mémoire, plafonds de la recherche web, tout se règle sans redéployer le code.
- **Agent web confiné** : un incident ou un comportement inattendu de l'agent n'affecte ni la base ni l'API principale.

5.4 Optimisation de l'index vectoriel : latence et pertinence

Dans le POC, la recherche reposait sur un index FAISS local de quelques milliers d'extraits. Dans le MVP, la recherche vectorielle est réalisée par PostgreSQL avec pgvector. La stratégie vise deux objectifs : garder une recherche rapide quand le volume augmente, et ne pas dégrader la pertinence des résultats. Les valeurs ci-dessous sont des points de départ, à valider par mesure pendant les jalons J2 (base vectorielle) et J3 (API RAG).

Levier	Choix proposé	Effet attendu
Type d'index	Recherche exacte comme référence de qualité. Index HNSW avec la distance cosinus activé dès le MVP, pour valider son comportement avant la montée en charge. Au volume actuel (environ 1 245 extraits), la recherche exacte est déjà rapide : l'index approximatif devient indispensable quand le volume croît.	Latence stable à mesure que le nombre d'extraits augmente
Réglages HNSW	Valeurs par défaut de pgvector au départ (m = 16, ef_construction = 64, ef_search = 40). On teste ensuite des valeurs d'ef_search plus élevées sur le jeu de questions annoté et on retient la plus basse qui préserve la pertinence.	Compromis latence / rappel choisi sur des mesures
Filtres	Filtre culturel, événements à venir et distance PostGIS appliqués dans la même requête SQL que la recherche par similarité. Si les filtres laissent trop peu de résultats, on augmente ef_search ou on utilise le balayage itératif de pgvector (versions récentes).	Résultats plus pertinents (proches, à venir, culturels) sans second appel
Nombre d'extraits	Entre 5 et 8 extraits transmis au modèle, après déduplication par identifiant d'événement. Découpage en extraits de 500 caractères avec chevauchement de 50 et en-tête répété, conservé depuis le POC.	Moins de tokens, donc réponse plus rapide et moins chère, sans perdre le contexte
Mise à jour	Ingestion quotidienne incrémentale : les nouveaux extraits s'ajoutent à l'index sans le reconstruire. Une reconstruction complète n'a lieu qu'en cas de changement de modèle d'embedding ou de réglage majeur, réalisée en parallèle puis basculée. Le dernier index valide est conservé si l'ingestion échoue.	Index à jour sans interruption de service

Monitoring

Niveau	Ce qui est suivi	Outil	Seuil d'alerte proposé
Technique	Latence p95, erreurs 5xx, CPU et mémoire, disponibilité	CloudWatch, alarmes, SNS	p95 supérieur à 6 s, taux d'erreur supérieur à 2 %
Qualité IA	Fidélité, pertinence, précision et rappel du contexte ; taux de refus ; part des requêtes envoyées au web	Ragas hebdomadaire sur le jeu annoté par un humain, table de résultats	Baisse nette d'un score par rapport à la semaine précédente
Données	Dernière ingestion réussie, nombre d'événements, part d'événements culturels, événements non localisés	Journal d'ingestion Kestra, requêtes SQL	Aucune ingestion réussie depuis 24 h
Coûts	Tokens par jour, appels de recherche web, coût par requête, budget AWS	Table d'interactions, AWS Budgets	Budget mensuel atteint à 80 %

6. Estimation des coûts build & OPEX

Tous les montants proviennent du classeur Excel (onglets Build, OPEX, Optimisations), recalculables à partir de l'onglet Hypothèses. Les prix cloud sont des estimations pour la région de Paris, à confirmer avec le simulateur AWS (Pricing Calculator) et par devis pour l'API de recherche web ; les coûts journaliers sont des hypothèses de travail. Les montants sont exprimés en euros avec un taux de change de 0,92 € pour 1 $.

6.1 Coût du développement initial (build)

Lot	Intitulé	Data Eng. (j)	Dév. (j)	DevOps (j)	Chef de projet (j)	Total (j)	Coût
L1	Socle cloud et CI/CD	2	1	6	1	10	5 180 €
L2	Données et base vectorielle	12	2	1	1	16	6 910 €
L3	API RAG et tests	4	8	0	1	13	6 040 €
L4	Mémoire conversationnelle et RGPD	3	4	0	1	8	3 720 €
L5	Contexte géographique	4	3	0	1	8	3 640 €
L6	Recherche web (smolagents)	2	7	0	1	10	4 760 €
L7	Monitoring et évaluation	5	0	2	1	8	3 700 €
L8	Interface Streamlit	0	5	0	1	6	3 000 €
L9	Sécurité, recette et charge	2	4	2	2	10	5 020 €
L10	Documentation et mise en production	3	0	0	2	5	2 400 €
L11	Pilotage de projet	0	0	0	8	8	4 800 €
	Total	**37**	**34**	**11**	**20**	**102**	**49 170 €**

Coûts journaliers retenus : Data Engineer 400 €, développeur 480 €, DevOps 550 €, chef de projet 600 €.

Récapitulatif du budget build	Montant
Main-d'œuvre	49 170 €
Aléas (15 % de la main-d'œuvre)	7 376 €
Cloud des environnements dev et staging (3 mois, taille réduite)	359 €
Outils, licences, nom de domaine	300 €
TOTAL BUILD (MVP)	**57 205 €**

6.2 Coût d'exploitation (OPEX) selon la charge utilisateurs

Trois scénarios de charge sont chiffrés, avec 8 requêtes par utilisateur et par mois, environ 2 500 tokens en entrée et 350 en sortie par requête, et 10 % des requêtes qui déclenchent la recherche web.

Poste	Pilote	Croissance	Passage à l'échelle
Utilisateurs actifs par mois	1 000	10 000	50 000
Requêtes par mois	8 000	80 000	400 000
Calcul (ECS Fargate : API, interface, agent, Kestra)	105 €	133 €	219 €
Base de données (RDS et stockage)	51 €	104 €	207 €
Réseau (load balancer, NAT Gateway, trafic)	52 €	64 €	149 €
Autres (S3, Secrets Manager, CloudWatch, WAF, DNS)	15 €	22 €	48 €
Mistral : réponses et embeddings	4 €	43 €	217 €
Agent web : modèle et API de recherche	5 €	54 €	272 €
Évaluation Ragas et mise à jour de l'index	7 €	7 €	7 €
TOTAL OPEX mensuel	**239 €**	**428 €**	**1 120 €**
TOTAL OPEX annuel	**2 873 €**	**5 132 €**	**13 434 €**
Coût par utilisateur actif et par mois	0,24 €	0,04 €	0,02 €
Coût par requête	0,030 €	0,005 €	0,003 €

Poste	Pilote	Croissance	Passage à l'échelle
Part des coûts variables (IA et API)	7 %	24 %	44 %

Lecture des résultats :

- Au pilote, l'OPEX est de **239 € par mois** et reste presque entièrement fixe (93 %) : on paie surtout la base de données, le calcul et le réseau, même avec peu d'utilisateurs.
- Avec la croissance, le coût par utilisateur chute (de 0,24 € à 0,04 € puis 0,02 €) : l'architecture est économiquement favorable à la montée en charge.
- À l'échelle, les coûts variables pèsent 44 % du total. L'**API de recherche web** devient le poste le plus sensible (206 € par mois), d'où l'intérêt du routeur, des plafonds et du cache.
- La NAT Gateway, souvent oubliée, pèse autant qu'une base de données de petite taille : c'est un des premiers leviers d'optimisation.

Coût de la première année (build + 12 mois d'OPEX)	Montant
Pilote	60 077 €
Croissance	62 336 €
Passage à l'échelle	70 639 €

6.3 Proposition d'optimisation budgétaire

Les leviers suivants sont chiffrés sur le scénario Croissance (OPEX de 428 € par mois). Comme ils se recouvrent en partie, un coefficient de prudence de 0,8 est appliqué.

ID	Levier	Taux	Économie par mois
O1	Remplacer Kestra par EventBridge Scheduler + tâche ECS planifiée L'ingestion quotidienne n'a pas besoin d'un orchestrateur permanent : une tâche planifiée suffit tant que le nombre de flux reste faible.	90 %	34 €
O2	Fargate sur processeurs Graviton (ARM) Mêmes conteneurs reconstruits en ARM64 : prix inférieur à performances équivalentes.	20 %	19 €
O3	Fargate Spot pour l'agent web Le service de repli peut tolérer une interruption : capacité Spot nettement moins chère.	60 %	11 €
O4	Supprimer la NAT Gateway Accès aux services AWS par VPC endpoints, règles de sécurité strictes pour les sorties vers Mistral et la recherche web.	70 %	27 €
O5	Cache des recherches web (taux porté à la cible) Mêmes questions fréquentes : une recherche déjà faite est servie depuis le cache.	29 %	12 €
O6	Cache sémantique des réponses Questions proches : la réponse précédente est réutilisée (durée de vie courte).	15 %	9 €
O7	RDS en instance réservée (1 an) Engagement de durée une fois la charge stabilisée (après 3 à 6 mois).	30 %	29 €
	Économie brute, puis nette après coefficient de prudence		**141 € puis 113 €**

Résultat : l'OPEX du scénario Croissance passe de **428 € à environ 315 € par mois (26 % d'économie)**.

Ordre de mise en œuvre conseillé : d'abord les gains rapides et sans risque (cache des recherches web, suppression de la NAT Gateway, Fargate sur Graviton), puis le remplacement de Kestra par une tâche planifiée si le nombre de flux reste faible, et enfin l'engagement sur la base de données une fois la charge stabilisée (3 à 6 mois). À mettre en place dès le départ : des alertes AWS Budgets, un plafond quotidien d'appels de recherche web et un suivi mensuel du coût par requête.

7. Bilan

7.1 Rappel des étapes clés du POC au MVP

Phase	Étapes clés	Statut
POC	Collecte des événements de Brest sur OpenAgenda, nettoyage et déduplication (697 événements)	Terminé et validé
	Découpage en chunks de 500 caractères (en-tête répété), vectorisation mistral-embed, index FAISS	Terminé
	Chatbot LangChain avec Mistral Small, instructions strictes contre l'invention, recherche dédupliquée	Terminé
	Tests automatisés (4 tests réussis, dont un test métier : événements récents et situés à Brest), README, rapport technique, soutenance	Terminé
	Jeu de test Ragas (15 questions, réponses de référence rédigées par l'équipe)	Jeu préparé, annotation humaine en cours
Étude MVP	Analyse des besoins, veille cloud et vectorielle, architecture, backlog, coûts, plan de projet (ce document)	Réalisé, à valider (J0)
MVP	J1 socle cloud et CI/CD, J2 données industrialisées, J3 API avec mémoire et géographie, J4 agent web et interface, J5 monitoring, J6 recette, J7 mise en production	Prévu du 2 nov. 2026 au 5 fév. 2027

7.2 Justification des choix techniques et méthodologiques

Choix	Justification	Alternative écartée
Garder LangChain et Mistral	Continuité avec le POC, abstractions qui permettent de changer de modèle ou de base sans réécriture	Changer de modèle ou de bibliothèque dès le MVP
PostgreSQL avec pgvector et PostGIS	Un seul magasin pour similarité, géographie, sessions et journaux ; volume faible ; compétence SQL de l'équipe	OpenSearch Serverless, Pinecone ; S3 Vectors gardé pour plus tard
ECS Fargate	Aucun serveur à gérer, déploiement simple, coût proportionné	Kubernetes (EKS) : trop lourd pour une petite équipe
Kestra	Déjà maîtrisé sur un projet précédent, planification et reprises sur erreur	Airflow ; EventBridge seul (gardé comme optimisation)
API FastAPI séparée de l'interface Streamlit	L'interface est remplaçable sans toucher à la logique ; l'API est testable seule	Tout dans une application Streamlit
Agent smolagents isolé, en repli	Limite le risque, le coût et la latence : le web n'est utilisé que si la base ne suffit pas	Agent web intégré à l'API principale, appelé à chaque question
Méthode : MoSCoW, jalons livrables, cycles de 2 semaines	Garantit un MVP utilisable à chaque jalon et protège le périmètre Must-Have	Planning linéaire sans démonstration intermédiaire
Estimations avec hypothèses explicites et marge de 15 %	Chiffres modifiables et discutables, aucun nombre caché	Estimation globale non détaillée

7.3 Défis rencontrés et solutions apportées

Défi	Solution apportée	Conséquence pour le MVP
Clé API Mistral écrite en dur dans un script du POC	Clé considérée comme compromise et régénérée ; usage d'un fichier .env et de python-dotenv	Secrets Manager et aucune clé dans le code ou les images
Résultats peu pertinents : le découpage séparait l'en-tête (titre, lieu, date) de la description	En-tête répété dans chaque chunk	Conserver cette structure lors de la migration vers pgvector
Le même événement revenait trois fois dans les résultats	Récupération de plus de candidats puis déduplication par identifiant d'événement	Déduplication à reprendre dans la requête SQL

Défi	Solution apportée	Conséquence pour le MVP
73 % des événements collectés étaient des offres d'emploi	Constat chiffré lors de l'analyse des données	Fonctionnalité F04 : filtrage culturel (Must-Have)
Aucun test ne vérifiait la logique métier (récent, à Brest)	Ajout d'un test métier et de la ville dans les colonnes conservées	Tests automatisés et jeu Ragas dans le pipeline CI/CD
Évaluer la qualité sans évaluer l'IA par l'IA	Jeu de questions dont les réponses attendues sont rédigées par un humain, juge Ragas fixe	Évaluation hebdomadaire sur ce jeu, enrichi à chaque incident
Concilier quatre axes, un budget et douze semaines	Priorisation Must-Have / Nice-to-Have, chiffrage par scénario de charge	Périmètre protégé, options reportées après les retours pilotes
Risque de sécurité et de coût de l'agent web	Isolation, outils limités, plafonds d'étapes et d'appels, cache	Garde-fous intégrés au périmètre Must-Have (F12, F13)

8. Conclusion

L'étude montre que le passage du POC au MVP est **réalisable en 12 semaines de travail** (fin prévue le 5 février 2027 avec la pause de fin d'année), pour un budget de construction d'environ **57 205 €** (102 jours de travail, aléas et environnements inclus), et un coût d'exploitation de **239 € à 1 120 € par mois** selon la charge, soit un coût par utilisateur qui diminue nettement avec la croissance. Une optimisation d'environ 26 % est accessible sans toucher aux fonctionnalités.

Les quatre axes demandés (mémoire, géographie, recherche web, monitoring) sont couverts par des fonctionnalités Must-Have, avec des garde-fous sur les deux points les plus risqués : l'agent web et les données personnelles. Le choix d'AWS, de PostgreSQL avec pgvector et PostGIS, et d'une architecture à quatre services indépendants s'appuie sur la veille et sur l'expérience de l'équipe, tout en restant réversible.

Décisions attendues de Jérémy au jalon J0 :

1. Valider le périmètre Must-Have, en particulier l'agent web en version minimale (repli uniquement).
2. Confirmer la disponibilité d'un développeur à temps partiel et d'un appui DevOps ponctuel (sinon, planning de 16 semaines).
3. Valider le choix d'AWS (région de Paris) et le principe d'une base PostgreSQL unique.
4. Autoriser l'obtention de devis pour l'API de recherche web et le chiffrage définitif avec le simulateur AWS.

9. Annexes

Lien vers le portfolio professionnel

Portfolio GitHub : https://yilmaz-ad.github.io/

Récapitulatif documenté des projets du parcours Data Engineer

Projet	Description et résultats	Outils et technologies	Lien GitHub
POC RAG Puls-Events (Projet 11)	Assistant de recommandation d'événements culturels à Brest. 697 événements collectés, environ 1 245 chunks vectorisés, recherche dédupliquée, 4 tests automatisés réussis. Démonstration : chatbot en ligne de commande répondant à partir des seules données OpenAgenda.	Python, pandas, LangChain, Mistral, FAISS, pytest, Ragas, python-dotenv	[Lien projet]
Forecast 2.0 GreenAndCoop	Pipeline ELT et déploiement cloud complet sur AWS (six étapes, de l'ingestion au suivi des coûts).	Airbyte sur EC2, dbt, PostgreSQL et Amazon RDS, Docker, ECR, ECS Fargate, EventBridge, CloudWatch	[Lien projet]

Projet	Description et résultats	Outils et technologies	Lien GitHub
BottleNeck (Projet 10)	Pipeline d'orchestration des flux : nettoyage et jointure de sources ERP et web, calculs de chiffre d'affaires, extractions CSV, exécution planifiée mensuelle.	Kestra, Python, DuckDB, YAML	[Lien projet]
Sport Data Solution (Projet 12)	Pipeline de bout en bout pour un programme d'avantages sportifs : 161 salariés, environ 6 900 activités simulées, 11 contrôles qualité sur 11 réussis, tableau de bord recalculable après modification des paramètres.	Debezium, Redpanda, PySpark, Delta Lake, PostgreSQL, Kestra, Power BI, Docker	[Lien projet]
Pipeline de tickets en temps réel	Traitement en continu de tickets, de l'ingestion au stockage, conteneurisé.	Redpanda, PySpark Structured Streaming, Docker Compose	[Lien projet]
Liens des 7 autres projets consultable sur le profile github			[Lien projet]

Compétences démontrées et valeur ajoutée

- **Conception de pipelines de données** : collecte d'API, nettoyage, déduplication, contrôles qualité, orchestration planifiée (Kestra, EventBridge).
- **IA générative appliquée** : architecture RAG, vectorisation, bases vectorielles, évaluation (Ragas), limitation des hallucinations.
- **Cloud et déploiement** : AWS (RDS, ECS Fargate, ECR, CloudWatch, EventBridge), conteneurs Docker, gestion des secrets, maîtrise des coûts.
- **Traitement en continu** : Redpanda, PySpark Structured Streaming, Delta Lake.
- **Gestion de projet** : cadrage des besoins, priorisation, backlog, estimation de coûts build et OPEX, planning par jalons, gestion des risques.
- **Rigueur et communication** : tests automatisés, documentation (README, runbooks), soutenances, justification des choix devant un mentor.

Liens externes et références utiles

- Veille cloud : Parts de marché 2026 (Artic Sledge)
- Veille vectorielle : Amazon S3 Vectors, disponibilité générale (AWS)
- Veille vectorielle : Comparatif des options sur AWS (GitHub)
- Agent web : smolagents, documentation Hugging Face
- Tarifs : Mistral API pricing (CloudZero)
- Tarifs : RDS db.t4g.medium (Bytebase)

Autres documents pertinents

Document	Contenu
MVP_Puls-Events_backlog_couts_planning.xlsx	Macro backlog (28 lignes), registre des risques, jalons, planning, build, OPEX par scénario, optimisations, hypothèses modifiables
evenements_brest.json	Données OpenAgenda nettoyées du POC (697 événements), utilisées pour l'analyse de la part d'offres d'emploi
test_ragas_dataset.json et evaluate_ragas.py	Jeu de 15 questions pour l'évaluation Ragas et script d'évaluation du chatbot
rapport_technique_puls_events.docx	Rapport technique du POC (architecture, choix, résultats)
README.md, chatbot.py, vectorize_data.py, data_extract.py, test_pipeline.py	Code et documentation du POC
soutenance_poc_rag.pptx	Support de la soutenance du POC (16 diapositives avec notes)